

Predictive GRU Trading Strategy for a Proprietary Trader

2113484

University of Bristol Business School

Abstract. This report implements a GRU-based predictive proprietary trader called PT2. PT2 aims to predict the future timestep 10 timesteps ahead, utilising a historical window of previous price data from the last 10 timesteps. PT2 further attempts to trade effectively in the BSE simulated market, aiming to make profitable trades. PT2 is found to perform inconsistently, with overall performance being worse than PT1, specifically due to volatility increases in market prices, making it difficult for the GRU model to effectively predict future share prices.

Keywords: Proprietary Trader · GRU · Bristol Stock Exchange · Limit Order Book.

1 Introduction

Algorithmic trading strategies have made a large impact on the financial world, completely changing the dynamics of modern exchanges. Predictive models are commonly used in trading strategies [3] as they give an indication of the future behaviour of the price of a certain asset. Having the ability to predict the future price of an asset remains a very complex and difficult practice, as much of the movement within asset prices is the result of random behaviour [4]. This gives rise to the issue of complexity that Artificial Neural Networks (ANNs) are designed to address. This report implements a predictive GRU trading strategy to predict the short-term future behaviour of share price, thereby making it possible to make trading decisions with a predictive edge.

2 Bristol Stock Exchange

Bristol Stock Exchange (BSE) [2] is a minimal simulation of a stock exchange market, designed to mimic the dynamics of a centralised electronic limit order book (LOB) for a single tradable security. BSE only allows a trader to have one order on the LOB at any given time, although that order can at any time be replaced by a new order. BSE does not represent a wholly accurate market scenario for these given reasons, additionally, BSE operates with zero latency in communication between traders and the exchange. Taking these factors into consideration, BSE does provide a testable simulation of an abstraction of a dynamic exchange, thereby enabling easy trader implementations and strategies.

3 Predictive GRU Trading Strategy

GRU (Gated Recurrent Unit) networks are a particular type of ANN, specifically a variant of a time-series Recurrent Neural Network (RNN), where previous time steps are considered at the current timestep to provide a prediction for future time steps. GRUs are specifically a variant of LSTMs that are designed to better handle long-term temporal dependencies compared to traditional RNNs, as RNNs suffer from the vanishing gradient problem [1], which results in them “forgetting” previous inputs. GRUs utilise gating mechanisms that regulate the flow of past information to better manage predictions.

4 Design of GRU Trading Strategy

4.1 GRU Prediction

The purpose of the GRU in this trading strategy is to take in a sequence of the last 10 time steps of price data from the tape, and then to predict the next one. This results in a model able to determine an estimate of what the price will be in the next time step, therefore providing information about what price to buy at, depending on whether the trader is buying or selling. This, therefore, provides a short-term forecast and does not resemble more complicated forecasting methods, where entire sequences of future data are predicted. Creating a model that generates future forecasts of entire sequences is inherently subject to growing uncertainty the further into the future that is predicted, and when jointly compared to the inherent randomness in price data, creates a very difficult problem to solve. Therefore, for simplicity, this report implements a single-timestep predictive method, thereby avoiding the issues of growing uncertainty and focusing on the error of single-timestep prediction.

The GRU designed for this report utilises a simple structure consisting of a single hidden layer of 50 neurons, with each of these neurons constituting a GRU unit. The goal of the GRU is to be a pricing model for the next timestep, and it simply needs to learn the behaviour of the short-term price fluctuations. This makes the GRU analogous to a moving average, although with added intelligence. As the task of the GRU is relatively straightforward, and long-term future forecasts do not need to be performed, the model is trained on a single sequence of BSE-generated tape data. This setup makes it possible for the GRU to provide real-time predictions when running in the trading simulation, as learning price data with different step sizes would produce inaccurate and indeterminable results. It is hypothesised in this paper that training the model with this single example of price data will constitute enough information to learn the general trend of price fluctuations.

4.2 GRU Trading Strategy

A simple strategy is implemented to utilise the GRU’s predictions. The trader is designed to alternate between buying and selling, only ever maintaining zero or

one units. The goal of the trader is to make short-term predictions of future price movements, and to capitalise on these price movements by executing trades, while also attempting to mitigate risk by determining when to make trades cautiously.

The current mid-price of the market is calculated from the best available bid and ask quotes. The predicted price is then compared to this current mid-price, from which a predicted return is calculated, which is the expected price change as a percentage over the next 10-timestep horizon. The PT2 trader then uses a threshold to determine when to buy or sell. When the trader is in buy mode, if the predicted return exceeds a positive threshold (set as a constant value), this would indicate a strong upward move of future price, therefore the trader attempts to buy. The trader does this by submitting a limit order bid of $\text{predicted_price} \times (1 - \text{margin})$. When the trader is in sell mode, if the predicted price negatively exceeds the negative threshold (same constant value), this indicates a significant drop in price, therefore the trader attempts to sell. This is done by submitting a limit order ask of $\text{predicted_price} \times (1 + \text{margin})$. This margin acts as a safety buffer for the trader, making the trades more conservative, only trading when trades are likely to be profitable. The trader, when in sell mode, also only trades when the ask price is higher than the last purchase price, to ensure it does not make a loss when selling. The only place the trader can make losses is when it buying, therefore relying on accurate predictions from the GRU.

Considering potential elements within this trader that may cause poor performance, the issue of volatility must be addressed. It is hypothesised that higher volatility will negatively impact the accuracy of the GRU’s predictions, therefore, the effect of higher volatility on the trader’s profits will be analysed through testing. Another issue relates to the trader’s prediction window, which relates to the forecast horizon. As this is set to 10 time steps ahead, the submitted trade may not get executed before the predicted price is reached, therefore not making use of the model’s predictive edge.

4.3 Hyperparameters and Tuning

The main hyperparameters to consider are those relevant to the GRU. To tune the GRU’s four most important hyperparameters, a grid search algorithm is conducted, where the GRU trains for 10 epochs on 16 different combinations of hyperparameters. The training loss is reported at the end, and the optimal set of hyperparameters is selected. These are shown in Table 4.1. The GRU hyperparameters are then set to the values shown, and the GRU is trained for 100 epochs. 100 epochs is chosen as the model converges within the first 10 time steps, therefore not requiring extensive training for longer than this. As this is a single-timestep predictive model, entire sequences do not need to be generated, therefore not requiring as extensive training.

Hidden Size	No. of Layers	Learning Rate	Batch Size	Training Loss
50	1	0.0005	32	0.0009736

Table 1. Selected GRU Hyperparameters. Selected by a grid search algorithm with 16 different combinations.

4.4 Computational Performance

Decisions were made to limit PT2 from becoming too computationally demanding. As PT2 imports a file of GRU hyperparameters and scalar values at each run, the prediction cannot take too long, as this would delay the time for the trader to make a decision, and may take longer than the forecast window, thereby completely losing its predictive edge. Therefore, the historical price window is maintained at 10 previous time steps, as this is the largest influence on model performance. This decision may have an impact on the predictive capabilities of the GRU, thereby reducing its capability, but tradeoffs need to be made to make the model work in a realistic way in the market.

5 Experiments

5.1 Experimental Setup and Metrics

Market setup conditions for these experiments are kept constant at the BSE default settings, specifically, trader splits and supply and demand schedules. The aim of these experiments is based on determining how changing specific market input data patterns, and analysing metrics from trials, can influence the performance of PT2 in comparison to PT1.

The tests are designed to specifically investigate how the GRU-based PT2 performs, considering the limitations of its architecture and logic. One of the most important considerations is volatility, as the GRU predictions become more uncertain under higher volatility conditions, but also potentially make more profits if the volatility is higher. Therefore, the experiment works by categorising a range of 50 historical 5-minute daily BTC files, and taking three files: one with the lowest volatility, one with the median volatility, and one with the highest volatility. This gives a good spread of volatility and aims to test the profits of PT2 in these contrasting conditions. Each of these three BTC files is used to run BSE for 100 trials on each.

Another important metric relevant to the performance of the GRU model is autocorrelation. Autocorrelation is a metric determining how dependent current price movements are on previous price movements. Since the GRU learns from sequential tape data, the presence of higher autocorrelation would result in the GRU being able to better predict the next price. This is because higher autocorrelation suggests that there are time-dependent structures within the data that the GRU can learn. It is hypothesised that a stronger autocorrelation in the data would improve the accuracy of the GRU predictions, thereby improving the trading decisions made by the PT2 trader. To test for this, a lag-1

autocorrelation measurement is taken from the results of the three sets of 100 trials. This works by calculating the Pearson’s correlation coefficient between successive price values within each trial.

To investigate the contributions of bad GRU predictions to PT2 performance, a mean-squared error (MSE) metric is used to determine error. This MSE metric is defined as

$$MSE = \frac{1}{N} \sum_{i=1}^N (Y_i - \hat{Y}_i)^2, \quad (1)$$

where N is the total number of samples, Y_i represents a true price value at point i in the sample, and $\hat{Y}_i$ represents a prediction at that same point.

5.2 Results

Looking at the results from the experiments, Figure 1 presents the distributions of PT1 and PT2 final balances, ranging over lower, median, and higher volatility BTC market days. In the lower volatility market, PT1 significantly outperforms PT2, with mean final balances of 2700 and 1867, respectively. This suggests that PT2 is not able to make large enough margins on the trades it is making, or the lack thereof. Similarly, PT1 outperforms PT2 in the higher volatility market, with means of 3016 and 1707, respectively. This suggests that the GRU predictions are being negatively affected by the extreme volatility, as the GRU has not seen this kind of behaviour before. Higher volatility markets also contain reduced autocorrelation, as the price movement is more like a random walk. Only with the median BTC market volatility do PT1 and PT2 have similar mean final profits, 2563 and 2456, respectively, suggesting PT2 can perform similarly to PT1. One major observation is the larger variability in final profits of PT2, suggesting it can sometimes make a lot more money than PT1, or a lot less money.

The underperformance of PT2 in both extremely high and low volatility markets shows the degradation in performance and suggests that the GRU-based predictions may underperform. Looking at predictive statistics, the MSEs for low, medium, and high volatilities were calculated as 22.05, 21.71, and 22.19, respectively. To interpret the true error in the prediction that corresponds to the PT2 trader’s real-time interpretation, these values need to be square-rooted to give prices. This yields values of 4.6958, 4.6596, and 4.7103, respectively. As price values fluctuate in the range of 100 to 200, this means the models have a 2-4% typical error relative to price level. This impacts PT2’s trading logic, as it tries to make small margins, therefore potentially having a large effect on performance. The median trials have the lowest MSE values, therefore corresponding to slight improvements in performance. Therefore, it can be noted that the GRU’s predictive capabilities are limited, requiring a reanalysis of trading logic to take these errors into account. It highlights the inherent difficulty in trying to predict patterns that are often dominated by a random process of trading, where only a few patterns can be learned to determine the next price.

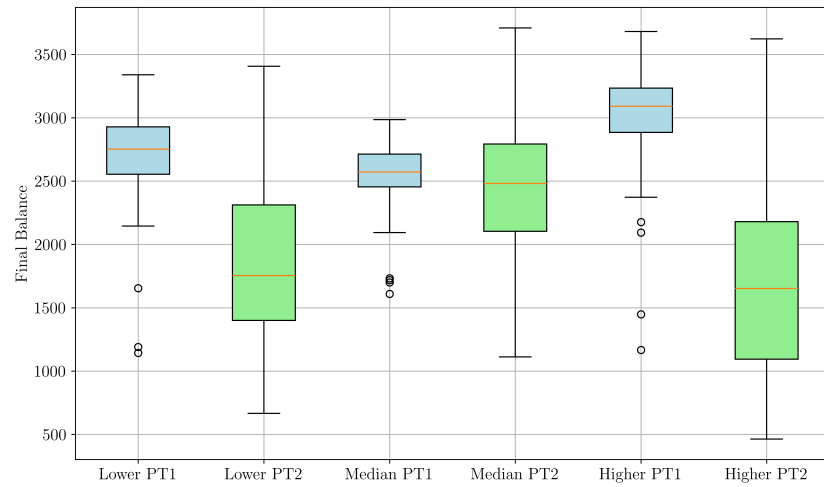


Fig. 1. Box plots of PT1 and PT2 final balances low, medium, and high volatility BTC markets.

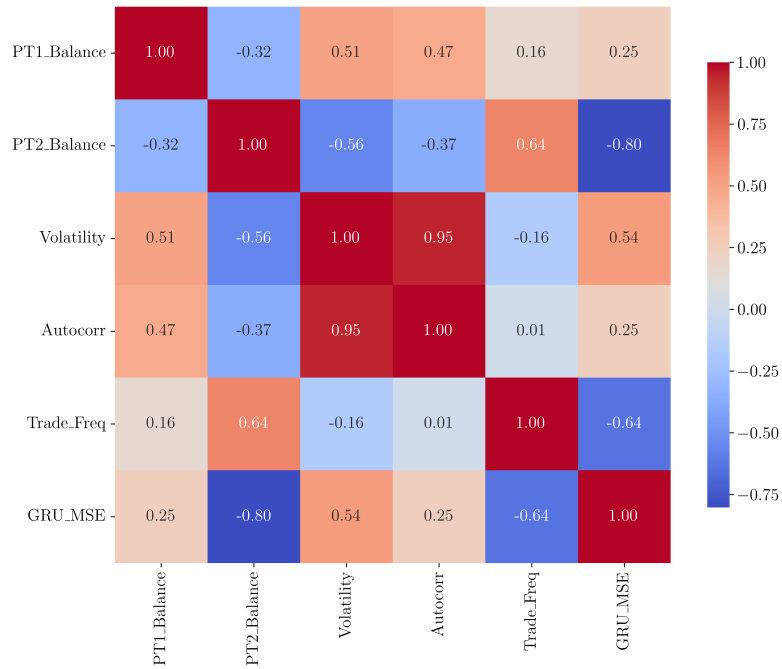


Fig. 2. Correlation Matrix of 300 trials. Summarising relationships between key metrics.

To effectively summarise the key metrics from all 300 combined trials, Pearson's correlation coefficient is calculated between all relevant metrics, and is shown in the correlation matrix in Figure 2. Many interesting insights can be drawn from this result. Firstly, the strongest negative correlation exists between PT2's final balance and the GRU MSE, being -0.80. This shows that as the predictive power of the GRU degenerates, increasing MSE, the total profit decreases. This supports the hypothesis that the performance of PT2 will partly rely on the predictive capabilities of the GRU, therefore showing an inherent weakness in this trading strategy. Further, a medium positive correlation of 0.54 is seen between the GRU's MSE and volatility.

Further, a medium negative correlation of -0.56 is shown between PT2's final balance and the volatility. This suggests that volatility may overall negatively impact the performance of PT2, even though in very low volatility conditions, PT2 also performs poorly. This suggests the optimal operating environment for PT2 is low to medium volatility conditions. Additionally, the correlation between PT2's final balance and autocorrelation shows a coefficient of -0.37. This contradicts the results for individual BTC files for 100 trials, where autocorrelation has a moderate positive impact, having correlation coefficients of around 0.65. The autocorrelation values for low, medium, and high volatility trials were calculated as very high values of 0.9549, 0.9360, and 0.9720, respectively. Considering the very strong correlation of 0.95 between volatility and autocorrelation for all 300 trials, and the high trial-by-trial autocorrelations, it suggests that the volatility may be the largest factor degrading GRU predictive performance, overwhelming any trial-based autocorrelation influences.

5.3 Statistical Analysis

To determine whether PT2 performs better than PT1, statistical tests need to be carried out. Firstly, the final balance distributions need to be checked for normality, for both traders. To do this, the Shapiro-Wilk test was carried out on all 300 trials, providing a p-value for normality for PT1 and PT2. The p-value needs to be >0.05 for the distributions to be considered normally distributed. The p-values for the final balance of PT1 and PT2 are 0.0000 and 0.0021, both being <0.05 . Therefore, both distributions are not normally distributed, requiring a non-parametric Wilcoxon signed-rank test, instead of a paired t-test.

The Wilcoxon signed-rank test is done in a paired manner with PT1 and PT2 balances, and requires its p-value to be <0.05 to reject the null hypothesis, being that there is no statistically significant difference between PT1 and PT2 balances. The p-value was calculated and found to be 0.0000, therefore being <0.05 , which means the null hypothesis can be rejected, with there being a statistically significant difference between the final balances of PT1 and PT2. The Wilcoxon signed-rank test does not determine which one is better; therefore, looking at the mean values and box plots in Figure 1, it can be determined that PT1 is better than PT2.

To test whether there is a clear difference in the effect that each BTC market volatility has on the final balance of PT2, a Kruskal-Wallis test is implemented,

as it does not require normality to be satisfied. This test determines if three or more independent groups differ significantly in their median values, with the three groups here being low, medium, and high volatility BTC markets. The test results in a p-value of around 0.0000, therefore being below the 0.05 significance level, thereby rejecting the null hypothesis that all groups are the same. Therefore, volatility is determined to impact PT2 performance.

6 Conclusion

This report explored the use of a single-timestep predictive GRU model to generate predictions of future prices to make effective trading decisions within the trader PT2. Through varying BTC input files, it was found that PT2 has similar performance to PT1 in the median volatility trials, while overall, for all trials, it was found that PT2 performs worse than PT1 statistically. Reasons for this were investigated, and a determination of performance degradation was made based on increased volatility, where the GRU model performed worse predictions with higher volatility trials. This showed a limiting factor within the implemented PT2 framework and suggests the requirement for more sophisticated and accurate models to improve PT2 trading performance. These findings suggest that predictive traders need to be trained on a larger set of varying volatility conditions to make accurate predictions. Overall, PT2 did make relatively consistent profits over most market conditions, therefore showing some promise in this implemented strategy.

References

1. Bengio, Y., Simard, P., Frasconi, P.: Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks* **5**(2), 157–166 (1994). <https://doi.org/10.1109/72.279181>
2. Cliff, D.: BSE: A Minimal Simulation of a Limit-Order-Book Stock Exchange. In: Affenzeller, M., et al. (eds.) *Proceedings of the 30th European Modeling and Simulation Symposium (EMSS 2018)*. pp. 194–203. DIME University of Genoa (2018)
3. Fischer, T., Krauss, C.: Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research* **270**(2), 654–669 (2018). <https://doi.org/https://doi.org/10.1016/j.ejor.2017.11.054>, <https://www.sciencedirect.com/science/article/pii/S0377221717310652>
4. Malkiel, B.G.: The efficient market hypothesis and its critics. *Journal of Economic Perspectives* **17**(1), 59–82 (March 2003). <https://doi.org/10.1257/089533003321164958>, <https://www.aeaweb.org/articles?id=10.1257/089533003321164958>

A PT2 Code

All important code files are saved on GitHub and can be accessed through this link: https://github.com/jacovaneeden/ACFIM0015_2113484.git [GitHub](#).


```

1 import pickle
2 import torch
3 import numpy as np
4 import torch.nn as nn
5 import pandas as pd
6
7 class GRUModel(nn.Module):
8     def __init__(self, input_size=1, hidden_size=50,
9         num_layers=1, output_size=1):
10         super(GRUModel, self).__init__()
11         self.hidden_size = hidden_size
12         self.num_layers = num_layers
13         self.gru = nn.GRU(input_size, hidden_size, num_layers
14             , batch_first=True)
15         self.fc = nn.Linear(hidden_size, output_size)
16
17     def forward(self, x):
18         # initialise hidden state
19         h0 = torch.zeros(self.num_layers, x.size(0), self.
20             hidden_size).to(x.device)
21         out, _ = self.gru(x, h0)
22         out = out[:, -1, :] # output from the last time step
23         out = self.fc(out)
24         return out
25
26 class TraderPT2(Trader):
27     """
28     Proprietary Trader Mark 2 (PT2) that uses a GRU model to
29     forecast future prices.
30
31     PT2 starts with a cash balance and utilises the same
32     bookkeeping and order management as PT1.
33
34     The main behaviour is as follows:
35     - PT2 maintains a history of the previous 10 trade price
36       from the exchange tape.
37     - When the history is full, it feeds the history into the
38       GRU model to predict a future price.
39
40     IMPORTANT: The GRU model must be set up exactly as shown
41       above, and the 'scalar.pkl' and 'gru_model.pt'
42       files need to be imported correctly for the GRU model to
43       work. This may require changing file paths.
44     """
45
46     def __init__(self, ttype, tid, balance, params, time):
47         # Initialize common trader variables

```

```

40     Trader.__init__(self, ttype, tid, balance, params,
41                     time)
42
43     self.job = 'Buy' # start with the
44                     intention to buy
45     self.last_purchase_price = None # will hold the
46                                     price of the asset when bought
47
48     if isinstance(params, dict):
49         if 'bid_percent' in params:
50             self.bid_percent = params['bid_percent']
51             if self.bid_percent > 1.0 or self.bid_percent
52                 < 0.01:
53                 sys.exit('FAIL: PT2 bid_percent=%f not in
54                     range [0.01,1.0]' % self.bid_percent
55                     )
56         if 'ask_delta' in params:
57             self.ask_delta = params['ask_delta']
58             if self.ask_delta < 0:
59                 sys.exit("FAIL: PT2 ask_delta can't be
60                     negative")
61         if 'n_past_trades' in params:
62             self.n_past_trades = int(round(params['
63                 n_past_trades']))
64             if self.n_past_trades < 1:
65                 sys.exit('FAIL: PT2 n_past_trades must be
66                     1 or more')
67
68     # load the trained GRU model
69     self.gru_model = GRUModel(input_size=1, hidden_size
70                               =50, num_layers=1, output_size=1)
71     try:
72         self.gru_model.load_state_dict(torch.load('
73             gru_model.pt'))
74     except Exception as e:
75         sys.exit("ERROR loading GRU model: " + str(e))
76     self.gru_model.eval() # set model to evaluation mode
77
78     # load the scaler from the pickle file
79     try:
80         with open("scaler.pkl", "rb") as f:
81             self.scaler = pickle.load(f)
82     except Exception as e:
83         sys.exit("ERROR loading scaler: " + str(e))
84
85     # the window size must match that used during
86         training of the GRU
87     self.window_size = 10
88     # initialise the rolling price history list
89     self.price_history = []

```

```

78
79 def update_price_history(self, lob):
80     """
81     Update the rolling list of normalised prices using
      the most recent Trade event from the exchange
      tape.
82     This method uses Pandas.
83     """
84     if 'tape' in lob and len(lob['tape']) > 0:
85         # look for the most recent trade event.
86         for event in reversed(lob['tape']):
87             if event.get('type') == 'Trade':
88                 # convert the price to a 2-D NumPy array
                  and normalise
89                 price_array = np.array([[event['price'
90                                         ]]], dtype=np.float32)
91                 norm_price = self.scaler.transform(
92                     price_array)[0][0]
93                 # append the new normalised price
94                 self.price_history.append(norm_price)
95                 # now create a Pandas Series from the
                  history
96                 price_series = pd.Series(self.
97                     price_history)
98                 # update the price_history with these
                  smoothed values
99                 self.price_history = price_series.tolist
100                 ()
101                 # keep only the most recent self.
                  window_size entries
102                 if len(self.price_history) > self.
103                     window_size:
104                     self.price_history = self.
105                         price_history[-self.window_size:]
106                 break # only update once per cycle.
107
108 def predict_price(self):
109     """
110     Feed the rolling window of normalised prices into the
      GRU model and return a predicted price
111     after converting the GRU output back using the scaler
      .
112     Returns None if there is insufficient history.
113     """
114     if len(self.price_history) < self.window_size:
115         return None # not enough data yet
116     # prepare input sequence: shape (batch=1,
117         sequence_length, input_size=1)
118     input_sequence = torch.tensor(self.price_history,
119         dtype=torch.float32).unsqueeze(0).unsqueeze(2)

```

```

112         with torch.no_grad():
113             predicted_norm = self.gru_model(input_sequence)
114             # convert the normalized prediction back to the
               original price scale
115         predicted_price = self.scaler.inverse_transform(
               predicted_norm)[0][0]
116         return predicted_price
117
118     def getorder(self, time, countdown, lob):
119         """
120         Return the current order for this trader as required
               by BSE.
121         In PT2 the order is stored in self.orders and is
               simply repackaged.
122         """
123         if countdown < 0:
124             sys.exit('Negative countdown')
125         if len(self.orders) < 1 or time < 5 * 60:
126             return None
127         else:
128             quoteprice = self.orders[0].price
129             order = Order(self.tid,
130                           self.orders[0].otype,
131                           quoteprice,
132                           self.orders[0].qty,
133                           time, lob['QID'])
134             self.lastquote = order
135             return order
136
137     def respond(self, time, lob, trade, vrbs):
138         # update price history for GRU input
139         self.update_price_history(lob)
140         # Get the GRU prediction.
141         predicted_price = self.predict_price()
142         confidence_threshold = 0.005 # 0.5% move
143         max_margin = 0.05
144
145         # calculate the current mid-price from the LOB
146         best_bid = lob['bids']['best']
147         best_ask = lob['asks']['best']
148         if best_bid is not None and best_ask is not None:
149             current_price = (best_bid + best_ask) / 2
150         elif best_bid is not None:
151             current_price = best_bid # only bids available
152         elif best_ask is not None:
153             current_price = best_ask # only asks available
154         else:
155             # no market info at all, skip trading this step
156             return
157

```

```

158     # check prediction availability
159     if predicted_price is None:
160         return # no valid prediction, skip
161
162     # calculate predicted return over the 20-step horizon
163     future_return = (predicted_price - current_price) /
164                     current_price
165     # Dynamically scale the margin based on expected
166     # return
167     # (bounded to avoid placing absurdly wide orders)
168     margin = min(max_margin, abs(future_return))
169
170     # BUY logic
171     if self.job == 'Buy' and lob['asks']['n'] > 0 and
172        predicted_price is not None:
173         if future_return > confidence_threshold:
174             bid_price = predicted_price * (1 - margin)
175             if bid_price < self.balance:
176                 order = Order(self.tid, 'Bid', bid_price,
177                               1, time, lob['QID'])
178                 self.orders = [order]
179
180     # SELL logic
181     elif self.job == 'Sell' and lob['bids']['n'] > 0 and
182        predicted_price is not None:
183         if future_return < -confidence_threshold:
184             ask_price = predicted_price * (1 + margin)
185             if ask_price > self.last_purchase_price:
186                 order = Order(self.tid, 'Ask', ask_price,
187                               1, time, lob['QID'])
188                 self.orders = [order]
189
190     def bookkeep(self, time, trade, order, vrbs):
191         """
192         Update trader bookkeeping after a trade is executed.
193
194         For a Buy trade, subtract the purchase price from the
195         cash balance and record the purchase.
196         For a Sell trade, add the sale price to the balance
197         and reset the held position.
198         Then remove the order from active orders.
199         """
200         mins = int(time // 60)
201         secs = time - 60 * mins
202         hrs = int(mins // 60)
203         mins = mins - 60 * hrs
204         outstr = 't=%f (%dh%02dm%02ds) %s (%s) bookkeep:
205                 orders=' % (time, hrs, mins, secs, self.tid, self
206                             .ttype)

```

```

198         for o in self.orders:
199             outstr += str(o)
200
201         self.blotter.append(trade)
202
203         transactionprice = trade['price']
204         if self.orders[0].otype == 'Bid':
205             # A successful bid (buy) reduces our cash balance
206             # and records the purchase price.
207             self.balance -= transactionprice
208             self.last_purchase_price = transactionprice
209             self.job = 'Sell'
210         elif self.orders[0].otype == 'Ask':
211             # A successful ask (sell) increases our cash
212             # balance.
213             self.balance += transactionprice
214             self.last_purchase_price = 0
215             self.job = 'Buy'
216         else:
217             sys.exit("FATAL: PT2 doesn't know order type %s"
218                     % self.orders[0].otype)
219
220         if vrbs:
221             net_worth = self.balance + (self.
222                 last_purchase_price if self.
223                 last_purchase_price else 0)
224             print('%s Balance=%d NetWorth=%d' % (outstr, self.
225                 .balance, net_worth))
226
227         # Remove the executed order.
228         self.del_order(order)
229
230 # end of PT2 definition

```